Python Programming Fundamentals

BankAccount Class

Programming Fundamentals Team

SETU

2026-06-12

Outline

Outline

1. Designing the BankAccount Class

Let's build a `BankAccount` class step by step.

What state does a bank account have?

- Account number
- Account holder name
- Current balance

1. Designing the BankAccount Class

Let's build a `BankAccount` class step by step.

What state does a bank account have?

- Account number
- Account holder name
- Current balance

What can a bank account do?

- Deposit money
- Withdraw money
- Get current balance
- Display account information

1. Designing the BankAccount Class

Let's build a `BankAccount` class step by step.

What state does a bank account have?

- Account number
- Account holder name
- Current balance

What can a bank account do?

- Deposit money
- Withdraw money
- Get current balance
- Display account information

This is the process of **object-oriented design**: identify the **attributes** (state) and **methods** (behaviour) before writing any code.

Outline

2. BankAccount — First Version

```
class BankAccount:
    """A simple bank account."""

    def __init__(self, account_no, name, initial_balance=0):
        self.account_no = account_no
        self.name       = name
        self.balance    = initial_balance

    def deposit(self, amount):
        """Add amount to balance."""
        self.balance += amount

    def withdraw(self, amount):
        """Subtract amount from balance."""
        self.balance -= amount
```

2. BankAccount — First Version

```
    def get_balance(self):
        """Return current balance."""
        return self.balance

    def __str__(self):
        return f"Account {self.account_no} ({self.name}):
€{self.balance:.2f}"
```

Outline

3. Creating and Using Instances

```python
# Create two separate accounts
alice_account = BankAccount("ACC001", "Alice Murphy", 500)
bob_account   = BankAccount("ACC002", "Bob Kelly")

# Each object has its own independent state
print(alice_account)    # Account ACC001 (Alice Murphy): €500.00
print(bob_account)      # Account ACC002 (Bob Kelly): €0.00

# Make some transactions
alice_account.deposit(200)
alice_account.withdraw(75)
bob_account.deposit(1000)
bob_account.deposit(500)
bob_account.withdraw(200)

print(alice_account)    # Account ACC001 (Alice Murphy): €625.00
```

3. Creating and Using Instances

```
print(bob_account)      # Account ACC002 (Bob Kelly): €1300.00

# Access balance directly
print(f"Alice's balance: €{alice_account.get_balance():.2f}")
```

Outline

4. Problems with the First Version

The current version has no validation:

```
acc = BankAccount("ACC003", "Carol", 100)

# These should all be rejected!
acc.withdraw(500)          # overdraft — balance becomes negative
acc.deposit(-50)           # negative deposit?!
acc.withdraw(-10)          # negative withdrawal adds money!

print(acc)   # Account ACC003 (Carol): €-440.00
```

4. Problems with the First Version

The current version has no validation:

```
acc = BankAccount("ACC003", "Carol", 100)

# These should all be rejected!
acc.withdraw(500)          # overdraft — balance becomes negative
acc.deposit(-50)           # negative deposit?!
acc.withdraw(-10)          # negative withdrawal adds money!

print(acc)   # Account ACC003 (Carol): €-440.00
```

We need to add validation:

- Deposit amount must be positive
- Withdrawal amount must be positive
- Cannot withdraw more than the current balance

Outline

5. BankAccount with Validation

```
class BankAccount:
    def __init__(self, account_no, name, initial_balance=0):
        if initial_balance < 0:
            raise ValueError("Initial balance cannot be negative")
        self.account_no = account_no
        self.name       = name
        self.balance    = initial_balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError(f"Deposit amount must be positive, got
{amount}")
        self.balance += amount
        print(f"Deposited €{amount:.2f}. New balance: €{self.balance:.2f}")

    def withdraw(self, amount):
```

5. BankAccount with Validation

```
        if amount <= 0:
            raise ValueError(f"Withdrawal amount must be positive, got
{amount}")
        if amount > self.balance:
            raise ValueError(f"Insufficient funds. Balance:
€{self.balance:.2f}")
        self.balance -= amount
        print(f"Withdrew €{amount:.2f}. New balance: €{self.balance:.2f}")
```

Outline

6. The __str__ Method

```
def __str__(self):
    return (f"BankAccount(\n"
            f"  Account No: {self.account_no}\n"
            f"  Holder:     {self.name}\n"
            f"  Balance:    €{self.balance:.2f}\n"
            f")")
```

6. The __str__ Method

6. The __str__ Method

```
    def __str__(self):
        return (f"BankAccount(\n"
                f"  Account No: {self.account_no}\n"
                f"  Holder:     {self.name}\n"
                f"  Balance:    €{self.balance:.2f}\n"
                f")")
```

Using the validated BankAccount:

```
acc = BankAccount("ACC001", "Alice", 1000)

acc.deposit(500)          # OK
acc.withdraw(200)         # OK

try:
    acc.withdraw(2000)    # raises ValueError
```

6. The __str__ Method

```
except ValueError as e:
    print(f"Error: {e}")

try:
    acc.deposit(-50)      # raises ValueError
except ValueError as e:
    print(f"Error: {e}")

print(acc)
```

Outline

7. Transaction History

Extending BankAccount with a transaction log:

```
class BankAccount:
    def __init__(self, account_no, name, initial_balance=0):
        self.account_no  = account_no
        self.name        = name
        self.balance     = initial_balance
        self.transactions = []      # list to hold history

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit must be positive")
        self.balance += amount
        self.transactions.append(("Deposit", amount, self.balance))

    def withdraw(self, amount):
```

7. Transaction History

```
        if amount <= 0:
            raise ValueError("Withdrawal must be positive")
        if amount > self.balance:
            raise ValueError("Insufficient funds")
        self.balance -= amount
        self.transactions.append(("Withdrawal", amount, self.balance))

    def print_statement(self):
        print(f"Statement for {self.name} ({self.account_no})")
        print("-" * 45)
        for t_type, amount, balance in self.transactions:
            print(f"  {t_type:<12} €{amount:>8.2f}   Balance:
€{balance:.2f}")
```

Thanks for Watching - Any questions?